

PRÁCTICA 1: SIMULACIÓN MCMC

Introducción a la inferencia bayesiana basada en simulación

El banco de datos `cars`, de `R`, contiene únicamente 2 variables: `speed` y `dist`. Los datos se corresponden con la velocidad (en millas por hora) a la que circulan una serie de coches y la distancia de frenado (medida en pies) necesaria para parar cada vehículo. Los datos fueron recogidos en 1920. Nuestra intención es intentar explicar, mediante un modelo de regresión lineal simple, bayesiano evidentemente, la distancia de frenado de los vehículos en función de su velocidad.

El modelo de regresión (bayesiano) que nos planteamos se corresponde con lo siguiente.

$$\begin{aligned}d_i &\sim N(\beta_0 + \beta_1 \cdot v_i, \sigma), \quad i = 1, \dots, n. \\ \beta_0, \beta_1 &\sim N(m, s) \\ \sigma &\sim \text{Uniforme}(0, u)\end{aligned}$$

donde d_i y v_i se corresponden, respectivamente, con las distancias de frenado y las velocidades. Las distribuciones normales de este modelo están parametrizadas en función de su desviación típica.

Lleva a cabo las siguientes tareas:

1. Determina valores para m , s y u , los hiperparámetros del modelo, de manera que las distribuciones iniciales propuestas sean no informativas.

Para que las distribuciones iniciales sean no informativas deberían ser básicamente planas sobre un rango de valores que comprendiera todos los posibles valores razonables que pudiera tomar la correspondiente variable. En ese caso podríamos tomar, por ejemplo, $m = 0$, $s = 1000$

lo que permitiría que tanto β_0 como β_1 pudiera tomar valores incluso de centenas de manera holgada, lo que parece más que suficiente. Por otro lado, podríamos tomar como límite superior para σ el valor $u = 1000$, que también parece suficientemente holgado para albergar cualquier valor razonable para este parámetro. Pero bueno, otras elecciones serían igualmente válidas ...

2. Desarrolla, y programa, un algoritmo de Gibbs para poder muestrear de la distribución a posteriori de los parámetros del modelo. Representa las distribuciones a posteriori que hayas estimado.¹

Empezamos calculando la función de verosimilitud:

$$\begin{aligned} L(\beta_0, \beta_1, \sigma) &= \prod_{i=1}^n P(d_i | \beta_0, \beta_1, \sigma) \propto \prod \left(\sigma^{-1} \exp \left(-\frac{(d_i - (\beta_0 + \beta_1 v_i))^2}{2\sigma^2} \right) \right) \\ &= \sigma^{-n} \exp \left(-\frac{\sum (d_i - (\beta_0 + \beta_1 v_i))^2}{2\sigma^2} \right) \end{aligned}$$

y, a partir de ella, la distribución conjunta a posteriori de los parámetros:

$$\begin{aligned} P(\beta_0, \beta_1, \sigma | \mathbf{d}) &\propto L(\beta_0, \beta_1, \sigma) \cdot P(\beta_0, \beta_1, \sigma) = L(\beta_0, \beta_1, \sigma) \cdot P(\beta_0) \cdot P(\beta_1) \cdot P(\sigma) \\ &\propto \sigma^{-n} \exp \left(-\frac{\sum (d_i - (\beta_0 + \beta_1 v_i))^2}{2\sigma^2} \right) \exp \left(-\frac{\beta_0^2}{2s^2} \right) \exp \left(-\frac{\beta_1^2}{2s^2} \right) I(\sigma < u) \end{aligned}$$

A partir de esta distribución conjunta podremos calcular la distribución condicional a posteriori de cada uno de los parámetros que nos permitirá implementar el algoritmo de Gibbs.

¹Si quieres puedes utilizar directamente los siguientes resultados para programar el algoritmo de Gibbs sampling:

$$\begin{aligned} P(\beta_0 | \mathbf{d}, \beta_1, \sigma) &\sim N \left(\frac{\sum (d_i - \beta_1 v_i)}{n + \sigma^2/s^2}, (n\sigma^{-2} + s^{-2})^{-1/2} \right), \\ P(\beta_1 | \mathbf{d}, \beta_0, \sigma) &\sim N \left(\frac{\sum ((d_i - \beta_0) \cdot v_i)}{\sum v_i^2 + \sigma^2/s^2}, (\sigma^{-2} \sum v_i^2 + s^{-2})^{-1/2} \right), \\ P(\sigma | \mathbf{d}, \beta_0, \beta_1) &\propto \sigma^{-n} \exp \left(-\frac{\sum (d_i - (\beta_0 + \beta_1 v_i))^2}{2\sigma^2} \right) I(\sigma < u). \end{aligned}$$

O si lo prefieres puedes deducir estas expresiones por ti mismo.

$$\begin{aligned}
P(\beta_0 \mid \mathbf{d}, \beta_1, \sigma) &\propto \exp\left(-\frac{\sum((d_i - \beta_1 \cdot v_i) - \beta_0)^2}{2\sigma^2}\right) \cdot \exp\left(-\frac{\beta_0^2}{2s^2}\right) \\
&\propto \exp\left(\frac{-1}{2} \cdot \left((n\sigma^{-2} + s^{-2})\beta_0^2 - 2\left(\sum \frac{d_i - \beta_1 v_i}{\sigma^2}\right)\beta_0\right)\right)
\end{aligned}$$

si tenemos en cuenta que el núcleo de una distribución normal de media μ y precisión τ tiene la siguiente forma:

$$P(x \mid \mu, \tau) \propto \exp\left(-\frac{\tau(x - \mu)^2}{2}\right) \propto \exp\left(-\frac{1}{2}(\tau x^2 - 2\tau\mu x)\right)$$

podemos deducir que la expresión a la que hemos llegado para $P(\beta_0 \mid \mathbf{d}, \beta_1, \sigma)$ se corresponde con una distribución Normal de precisión $n\sigma^{-2} + s^{-2}$ y de media $(\sum \frac{d_i - \beta_1 v_i}{\sigma^2}) / (n\sigma^{-2} + s^{-2})$. Así, en términos de la desviación típica tenemos que:

$$P(\beta_0 \mid \mathbf{d}, \beta_1, \sigma) \sim N\left(\frac{\sum(d_i - \beta_1 v_i)}{n + \sigma^2/s^2}, (n\sigma^{-2} + s^{-2})^{-1/2}\right)$$

De manera similar calculamos la distribución condicional a posteriori de β_1 :

$$\begin{aligned}
P(\beta_1 \mid \mathbf{d}, \beta_0, \sigma) &\propto \exp\left(-\frac{\sum((d_i - \beta_0) - \beta_1 v_i)^2}{2\sigma^2}\right) \exp\left(-\frac{\beta_1^2}{2s^2}\right) \\
&= \exp\left(\frac{-1}{2} \left((\sigma^{-2} \sum v_i^2 + s^{-2})\beta_1^2 - 2\sigma^{-2} \sum((d_i - \beta_0) \cdot v_i)\beta_1\right)\right)
\end{aligned}$$

que se corresponde con una distribución Normal de precisión $\sigma^{-2} \sum v_i^2 + s^{-2}$ y media $\sum((d_i - \beta_0) \cdot v_i) / (\sum v_i^2 + \sigma^2/s^2)$ o, en términos de su desviación típica:

$$P(\beta_1 \mid \mathbf{d}, \beta_0, \sigma) \sim N\left(\frac{\sum((d_i - \beta_0) \cdot v_i)}{\sum v_i^2 + \sigma^2/s^2}, (\sigma^{-2} \sum v_i^2 + s^{-2})^{-1/2}\right)$$

Por último, calculamos la distribución condicional a posteriori de σ :

$$P(\sigma \mid \mathbf{d}, \beta_0, \beta_1) \propto \sigma^{-n} \exp\left(-\frac{\sum(d_i - (\beta_0 + \beta_1 v_i))^2}{2\sigma^2}\right) I(\sigma < u)$$

Esta distribución, no se corresponde con ninguna de las distribuciones habituales, de hecho se trata de una distribución truncada por debajo del valor u , por tanto no dispondremos de ninguna función de **R** capaz de generar valores de esta distribución. De esta manera nos vemos forzados a simular de σ mediante el algoritmo de Metropolis-Hastings.

Una vez disponemos de las distribuciones condicionales a posteriori de todos los parámetros del modelo podemos desarrollar, y programar en **R**, el algoritmo de Gibbs correspondiente:

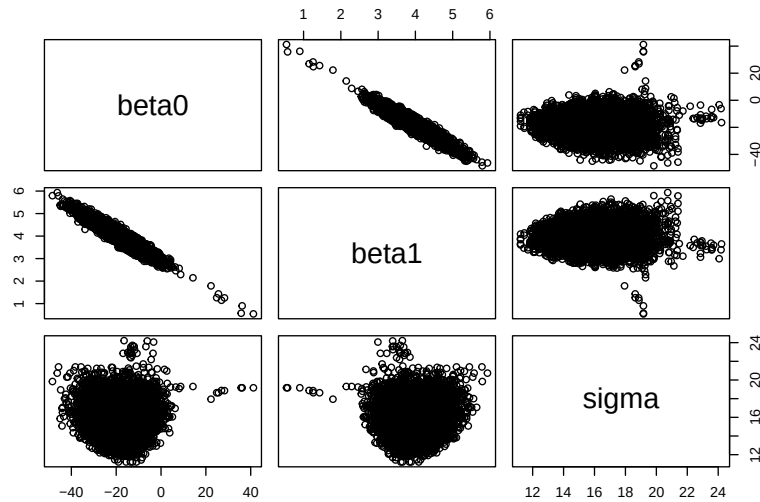
```

d <- cars$dist; v <- cars$speed; n <- dim(cars)[1]
m <- 0; s <- 1000; u <- 1000
samples <- matrix(nrow = 10000, ncol = 3)
colnames(samples) <- c("beta0", "beta1", "sigma")
beta0 <- mean(d); beta1 <- 0; sigma <- 20
# Probabilidad a posteriori de sigma
P.sigma <- function(sigma){
  sigma^(-n) * exp(-sum((d-(beta0+beta1*v))^2)/(2*sigma^2))*(sigma<u)
}
# Contador para controlar el porcentaje de veces que se aceptan los
# valores propuestos en el algoritmo de Metropolis para sigma
cuenta.acepta <- 0
# Longitud del salto para el muestreo de sigma por Metropolis
delta <- 1
set.seed(1)
for(i in 1:(nrow(samples))){
  beta0 <- rnorm(1, sum(d-beta1*v)/(n+(sigma/s)^2), (n/(sigma^2)+s^(-2))^(-0.5))
  beta1 <- rnorm(1, sum((d-beta0)*v)/(sum(v^2)+(sigma/s)^2),
    ((sum(v^2)/(sigma^2))+s^(-2))^(-0.5))
  sigma.star <- rnorm(1, sigma, delta)
  acepta <- rbinom(1, 1, min(P.sigma(sigma.star)/P.sigma(sigma),1))
  if(acepta == 1){
    sigma <- sigma.star
    cuenta.acepta <- cuenta.acepta + 1
  }
  samples[i,] <- c(beta0, beta1, sigma)
}

# Representación, para cada variable, de la muestra a posteriori obtenida.

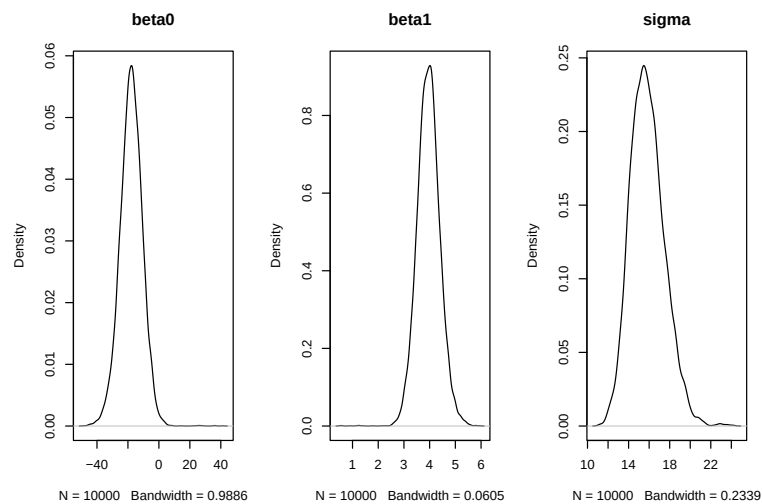
```

```
pairs(samples)
```



Representación de la función de densidad a posteriori para cada variables.

```
par(mfrow=c(1,3))
plot(density(samples[,1]),main="beta0")
plot(density(samples[,2]),main="beta1")
plot(density(samples[,3]),main="sigma")
```



Probabilidad de aceptación para sigma en el algoritmo de Metropolis.

```
cuenta.acepta/10000
```

```
## [1] 0.8121
```

```
# toma un valor un poco alto por lo que podríamos utilizar un salto (delta) algo
# más largo. Debería mejorar la convergencia del modelo.
```

3. Compara los resultados obtenidos de tu simulación con los de un modelo lineal frecuentista (función `lm` de R) ¿son similares?

```
# Resumen de los resultados bayesianos, mediante MCMC.
summary(samples[-(1:500), ])
```

```
##      beta0      beta1      sigma
## Min.   :-48.429  Min.    :2.549  Min.    :11.22
## 1st Qu.: -22.515  1st Qu.:3.672  1st Qu.:14.68
## Median :-17.882  Median :3.955  Median :15.73
## Mean   :-18.005  Mean    :3.961  Mean    :15.83
## 3rd Qu.: -13.269  3rd Qu.:4.237  3rd Qu.:16.86
## Max.    :  6.281  Max.    :5.796  Max.    :21.48
```

```
# Resultados frecuentistas.
resul.lm <- lm(dist ~ speed, data = cars)
summary(resul.lm)
```

```
##
## Call:
## lm(formula = dist ~ speed, data = cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -29.069  -9.525  -2.272   9.215  43.201
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -17.5791     6.7584  -2.601  0.0123 *
```

```
## speed          3.9324      0.4155    9.464 1.49e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.38 on 48 degrees of freedom
## Multiple R-squared:  0.6511, Adjusted R-squared:  0.6438
## F-statistic: 89.57 on 1 and 48 DF,  p-value: 1.49e-12

# Ambos resultados parecen coincidir.
```

4. Utiliza la distribución a posteriori que has simulado para estimar el coeficiente de determinación R^2 del modelo. Obtén tanto un estimador puntual de este valor como un intervalo de credibilidad al 95 %.

El coeficiente de determinación de un modelo se define como: $1 - \sigma^2 / \text{Var}(\mathbf{y})$ donde σ^2 es la varianza residual del modelo y $\mathbf{y}$ la variable respuesta del modelo. En ese caso podríamos obtener una muestra de la distribución a posteriori de este estadístico mediante:

```
R2 <- 1 - (samples[, 3]^2)/var(cars$dist)
# Estimador puntual (media a posteriori) de R^2
mean(R2)

## [1] 0.6182982

# Intervalo de credibilidad
quantile(R2, c(0.025,0.975))

##      2.5%      97.5%
## 0.4352890 0.7474081
```